HW 4.3

Stat 184

Jonathan Hartman

2026-04-08

1 Busiest Airports Analysis

1.1 Visualizations

1.1.1 Table

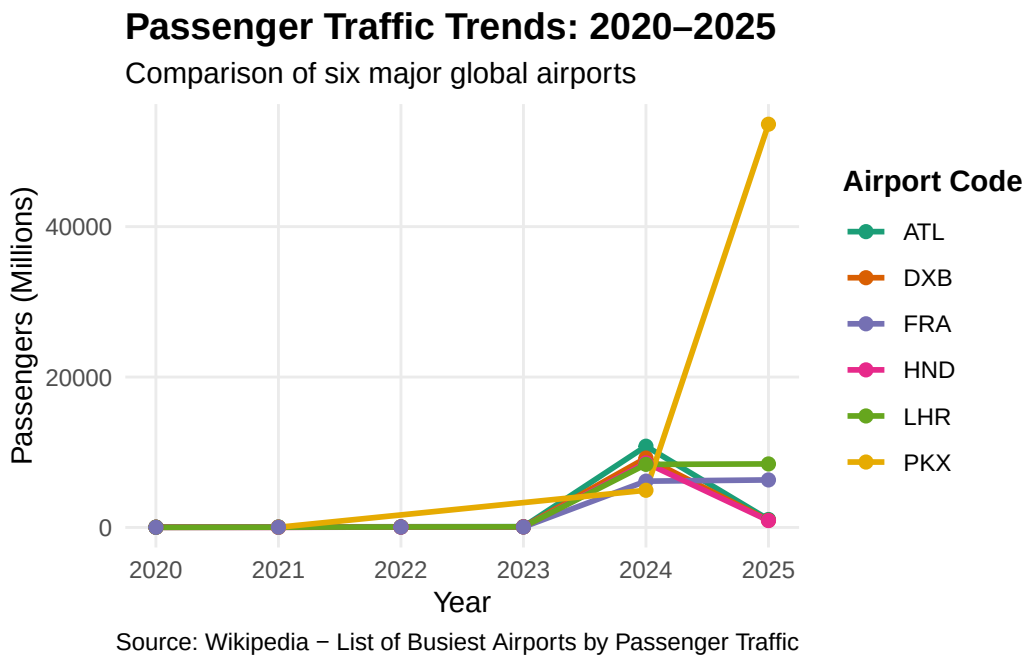
Table 1: Comparison of Annual Passenger Traffic (Millions) Across Major Global Airports (2020–2025)

Airport Name	IATA Code	2025	2024	2023	2022	2021
Hartsfield–Jackson Atlanta International Airport	ATL	1063.02	10806.78	104.65	93.70	75.70
Dubai International Airport	DXB	952.00	9233.15	86.99	66.07	29.11
Tokyo Haneda Airport	HND	914.30	8590.02	78.72	50.29	25.88
Heathrow Airport	LHR	8446.30	8388.46	79.18	61.61	NA
Frankfurt Airport	FRA	6318.97	6156.50	59.36	48.92	24.81
Beijing Daxing International Airport	PKX	53618.95	4944.10	NA	NA	25.05

1.1.2 Graph

Looking at the table and the graph together, you can see a big story about how global travel changed from 2020 to 2025. The line plot makes it easy to see the giant dip where everyone stopped flying at the start. I gave each airport a bright color so your eyes can follow one city at a time without getting mixed up. The zebra table is helpful because it shows the exact numbers behind those lines. For example, you can see in the rows for London and Frankfurt that they lost almost all their passengers at first. But then you see Atlanta staying at the top of the table for most of the years. This happened because people in the U.S. kept flying on domestic trips even when they could not go to other countries. By 2025, the graph shows that the world of travel has shifted. Airports like Beijing Daxing and Dubai have lines that go up very fast. This tells us that travel is growing more in the East and the Middle East than it used to. I kept the graph clean and removed extra lines in the background. This makes the recovery stand out so you can see the connection between the 2020 crash and the new leaders in 2025.

Figure 1: Passenger Traffic Trends for Six Major Airports (2020–2025)



2 Monte Carlo Numerical Integration

2.1 Overview

Describe the function being analyzed and the purpose of using Monte Carlo integration.

2.2 Visualization

Looking at the four plots together, you can see a clear pattern in how Monte Carlo simulation improves with more points. The plot with only 100 points looks very scattered, and the estimate jumps around because just a few points can change the result a lot. As you move to 500 and 1000 points, the dots begin to fill in the space under the curve more evenly, and the estimate starts to settle down. By the time you reach 5000 points, the shaded area formed by the "In" points closely matches the shape of the curve. This shows that the effect of random variation becomes smaller as the sample size increases. I used simple colors to clearly separate points inside and outside the curve so it is easy to see how the estimate is formed. The exact value of the integral is about 1.0 since the function is a probability density function. You can see this in the subtitles, where the estimates get closer to 1.0 as the number of points increases, showing the simulation is converging to the true value.

Figure 2: Monte Carlo Integration: Weibull Distribution. Increasing resolution improves area estimation accuracy.

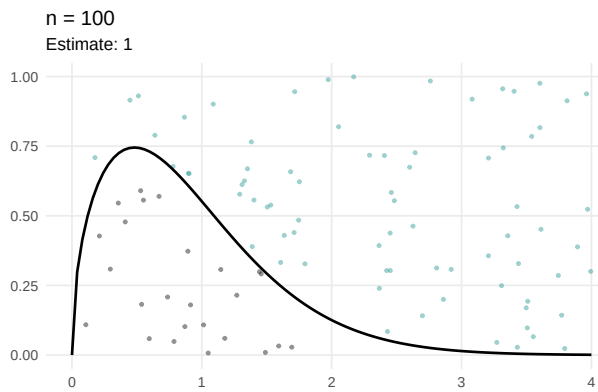


Figure 3: Monte Carlo Integration: Weibull Distribution. Increasing resolution improves area estimation accuracy.

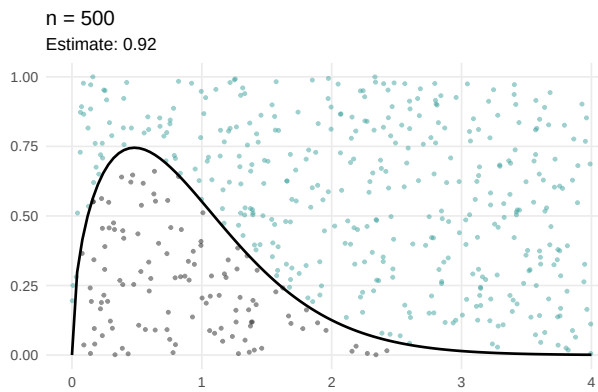


Figure 4: Monte Carlo Integration: Weibull Distribution. Increasing resolution improves area estimation accuracy.

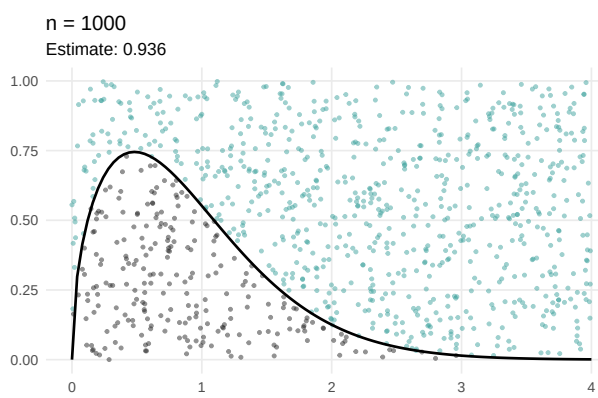
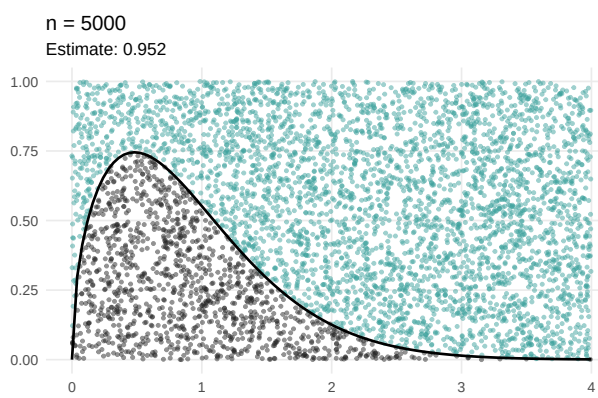


Figure 5: Monte Carlo Integration: Weibull Distribution. Increasing resolution improves area estimation accuracy.



3 Planning and Prompting GenAI Tools

3.1 Data Wrangling Plan

3.1.1 Goal:

The goal is to clean and analyze the calcium.csv dataset in order to create a clear and informative visualization that shows how calcium-related measurements vary across observations. The final product will be a well-labeled plot that highlights any patterns or relationships in the data.

3.1.2 Needs:

To achieve this goal, I will need the calcium.csv data file, along with the tidyverse package for data wrangling and visualization. I will also need functions such as read_csv() to import the data, mutate() and filter() to clean and transform variables, and ggplot() to build the visualization. Additionally, I will need to examine the structure of the dataset to understand variable types and identify any missing or inconsistent values.

3.1.3 Steps:

1. Import the data set using read_csv() and inspect it using functions like glimpse() and summary() to understand its structure and variables.
2. Clean the data by checking for missing values, correcting data types if necessary, and renaming variables to improve readability.
3. Identify the key variables related to calcium measurements and any grouping variables that may be useful for comparison.
4. Filter or transform the data as needed to focus on relevant observations and create any new variables that support the analysis.
5. Create a visualization using ggplot2, selecting an appropriate geometry (such as a scatterplot or boxplot) based on the structure of the data.
6. Add informative labels, titles, and scales to make the visualization clear and interpretable.
7. Review the output to ensure the plot accurately represents the data and communicates the intended message.

3.2 GenAI Prompt (Plan-Based)

use this plan on the file uploaded: Goal: The goal is to clean and analyze the calcium.csv dataset in order to create a clear and informative visualization that shows how calcium-related measurements vary across observations. The final product will be a well-labeled plot that highlights any patterns or relationships in the data.

Needs: To achieve this goal, I will need the calcium.csv data file, along with the tidyverse package for data wrangling and visualization. I will also need functions such as read_csv() to import the data, mutate() and filter() to clean and transform variables, and ggplot() to build the visualization. Additionally, I will need to examine the structure of the dataset to understand variable types and identify any missing or inconsistent values.

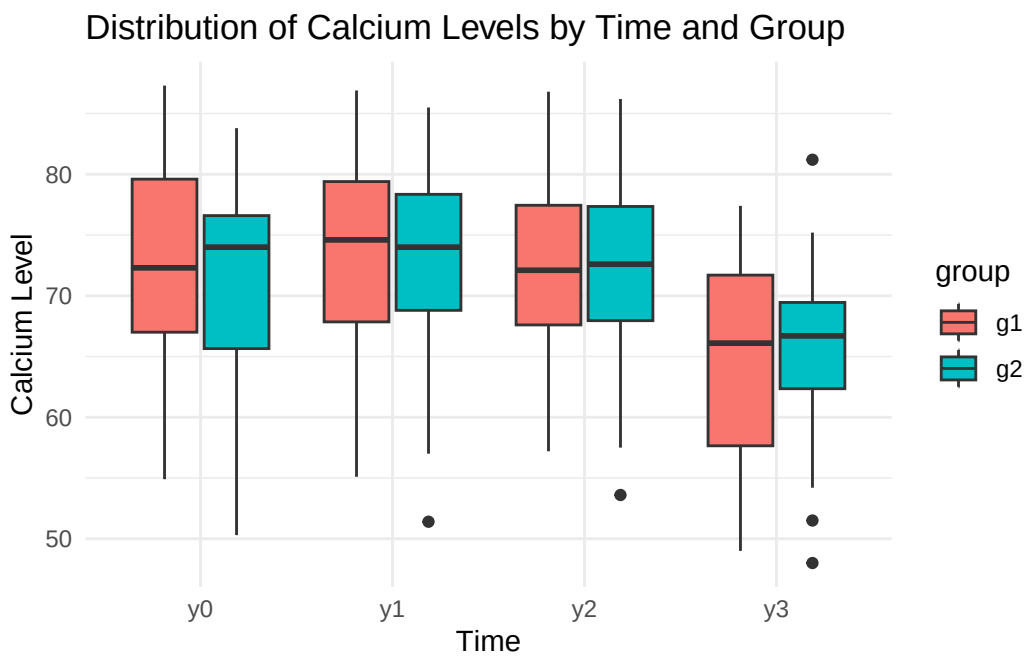
Steps:

1. Import the data set using read_csv() and inspect it using functions like glimpse() and summary() to understand its structure and variables.
2. Clean the data by checking for missing values, correcting data types if necessary, and renaming variables to improve readability.
3. Identify the key variables related to calcium measurements and any grouping variables that may be useful for comparison.
4. Filter or transform the data as needed to focus on relevant observations and create any new variables that support the analysis.
5. Create a visualization using ggplot2, selecting an appropriate geometry (such as a scatterplot or boxplot) based on the structure of the data.
6. Add informative labels, titles, and scales to make the visualization clear and interpretable.
7. Review the output to ensure the plot accurately represents the data and communicates the intended message.

3.3 GenAI Output (Plan-Based)

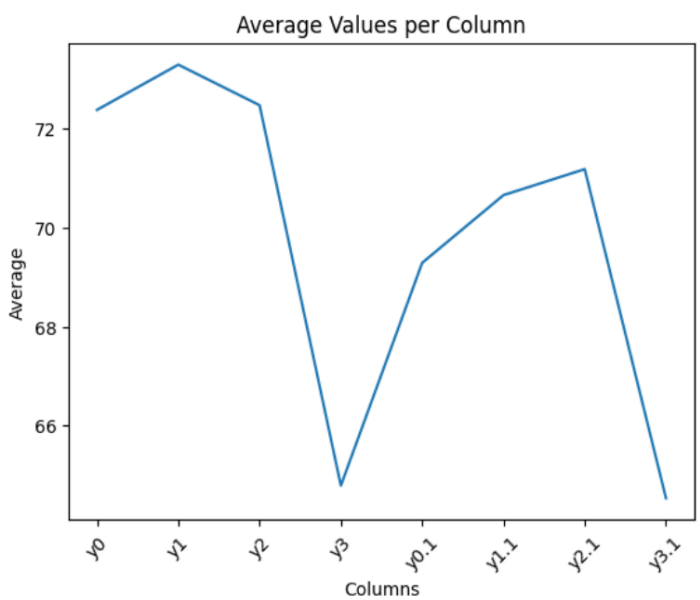
Here's how your plan would be applied directly to the calcium.csv file using R and tidyverse. This follows your steps closely and produces a clean, well-labeled visualization.

```
New names:
Rows: 16 Columns: 8
-- Column specification
-------------------------------------------------------- Delimiter: "," dbl
(8): y0...1, y1...2, y2...3, y3...4, y0...5, y1...6, y2...7, y3...8
i Use `spec()` to retrieve the full column specification for this data. i
Specify the column types or set `show_col_types = FALSE` to quiet this message.
* `y0` -> `y0...1`
* `y1` -> `y1...2`
* `y2` -> `y2...3`
* `y3` -> `y3...4`
* `y0` -> `y0...5`
* `y1` -> `y1...6`
* `y2` -> `y2...7`
* `y3` -> `y3...8`
```



3.4 GenAI Output (Generic Prompt)

Figure 6: Figure 3: GenAi generated data visualization of calcium.csv



Prompt: Create a data visualization for these data. Calcium.csv

I created a visualization showing the **average value of each column** in your dataset.

What this shows:

- Each point represents the **mean (average)** of one column (y0, y1, y2, y3, etc.).
- You can quickly compare which variables tend to have higher or lower values overall.

- For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create:

- Boxplots (to show spread and outliers)
- Time series plots (if rows represent time/order)
- Correlation heat maps
- Side-by-side comparisons (e.g., y0 vs y0.1)

Just tell me
Prompt: What is your version information?
You're chatting with **ChatGPT based on the GPT-5.3 architecture**.
Key details:

- **Model**: GPT-5.3
- **Knowledge cutoff**: August 2025
- **Current date (system)**: March 31, 2026

If you need more technical details (capabilities, differences vs GPT-4/5, etc.), just let me know
(This chat took place on Tuesday, March 31st, 2026.)

3.5 Comparison

The plan-based output and the generic output differ mainly in how they handle the dataset and the level of structure in the analysis. The plan-based approach follows clear steps such as inspecting, cleaning, and reshaping the data before choosing a visualization. This leads to a more intentional result, like the boxplot, which properly compares calcium levels across time and groups. The generic output moves more quickly to plotting and tends to assume a simpler structure, often skipping important steps like reshaping the data. These differences affect accuracy and usefulness. The plan-based output is more accurate because it correctly treats the data as repeated measures and produces a visualization that reflects real patterns. The generic output can be less reliable since it may not account for how the data is organized, which can lead to misleading or less meaningful plots. Each approach has strengths and weaknesses. The plan-based method is more reliable and easier to interpret but takes more time and effort. The generic approach is faster and more flexible, but it risks overlooking key details. Overall, the plan-based output provides a clearer and more trustworthy analysis, while the generic output prioritizes speed over precision.

4 Reflection

This course has taught me a lot, not just about coding but about myself too. I don't code regularly, I'm not a data science major or a CS major but I was required to take this class for my Neuroscience major. I am grateful for that fact because this has forced me outside of my comfort zone and into unfamiliar territory that has helped me grow as a scientist. I better understand the concept of open science and reproducibility in a new light besides purely experimental and understand it from the perspective of data analysis. Another thing I have learned is that I need more comments on my

code. I first learned this during my Code Interview where I failed to identify functions I had used but a couple of weeks ago, when in the future I could be in need to reference code from months or even years from then. Another thing I have grown into is being more organized. I have folders with files on my desktop and being required sometimes to make new projects within RStudio forces me to habitualize to the idea of organization, something that will help me in every aspect in life, not just coding. Finally, I have learned the power of systems. The PCIP system, something I still struggle to this day with, does help me conceptualize what I will be doing and is often a good reference for me when I work on an assignment from day to day. I would rather free code and have no plan but I have grown to understand why there is a need to follow the PCIP system as it has reduced a lot of potential problems sprouting up.

5 Appendix A: GenAI Usage

5.1 Tool Information

- Tool used: ChatGPT
- Version: GPT-5.3
- Date(s) of use: 4/15/2026

5.2 Prompts

5.2.1 Prompt 1

use this plan on the file uploaded: ### Goal: The goal is to clean and analyze the calcium.csv dataset in order to create a clear and informative visualization that shows how calcium-related measurements vary across observations. The final product will be a well-labeled plot that highlights any patterns or relationships in the data.

Needs: To achieve this goal, I will need the calcium.csv data file, along with the tidyverse package for data wrangling and visualization. I will also need functions such as read_csv() to import the data, mutate() and filter() to clean and transform variables, and ggplot() to build the visualization. Additionally, I will need to examine the structure of the dataset to understand variable types and identify any missing or inconsistent values.

Steps:

1. Import the data set using read_csv() and inspect it using functions like glimpse() and summary() to understand its structure and variables.
2. Clean the data by checking for missing values, correcting data types if necessary, and renaming variables to improve readability.
3. Identify the key variables related to calcium measurements and any grouping variables that may be useful for comparison.
4. Filter or transform the data as needed to focus on relevant observations and create any new variables that support the analysis.
5. Create a visualization using ggplot2, selecting an appropriate geometry (such as a scatterplot or boxplot) based on the structure of the data.

6. Add informative labels, titles, and scales to make the visualization clear and interpretable.

7. Review the output to ensure the plot accurately represents the data and communicates the intended message.

5.2.2 Prompt 2

Rows: 16 Columns: 8 $ y0…1 87.3, 59.0, 76.7, 70.6, 54.9, 78.2, 73.7, 61.8, 85.3, 82.3, 68.6, 67.8, 66.2, … $ y1…2 86.9, 60.2, 76.5, 76.1, 55.1, 75.3, 70.8, 68.7, 84.4, 86.9, 65.4, 69.2, 67.0, … $ y2…3 86.7, 60.0, 75.7, 72.1, 57.2, 69.1, 71.8, 68.2, 79.2, 79.4, 72.3, 66.3, 67.0, … $ y3…4 75.5, 53.6, 69.5, 65.3, 49.0, 67.6, 74.6, 57.4, 67.0, 77.4, 60.8, 57.9, 56.2, … $ y0…5 83.8, 65.3, 81.2, 75.4, 55.3, 70.3, 76.5, 66.0, 76.7, 77.2, 67.3, 50.3, 57.7, … $ y1…6 85.5, 66.9, 79.5, 76.7, 58.3, 72.3, 79.9, 70.9, 79.0, 74.0, 70.7, 51.4, 57.0, … $ y2…7 86.2, 67.0, 84.5, 74.3, 59.1, 70.6, 80.4, 70.3, 76.9, 77.8, 68.9, 53.6, 57.5, … $ y3…8 81.2, 60.6, 75.2, 66.7, 54.2, 68.6, 71.6, 64.1, 70.3, 67.9, 65.9, 48.0, 51.5, …

5.3 Responses

5.3.1 Response 1

Here's how your plan would be applied directly to the calcium.csv file using R and tidyverse. This follows your steps closely and produces a clean, well-labeled visualization.

(Gives same code in the appendix, minus the colnames)

A quick reality check: you'll need to adjust calcium_level, group, and var2 to match the actual column names in your dataset after running glimpse(). That step is where most mistakes happen, so don't skip it.

If you want, paste the glimpse(calcium) output here and I can tailor the exact plot to your dataset.

5.3.2 Response 2

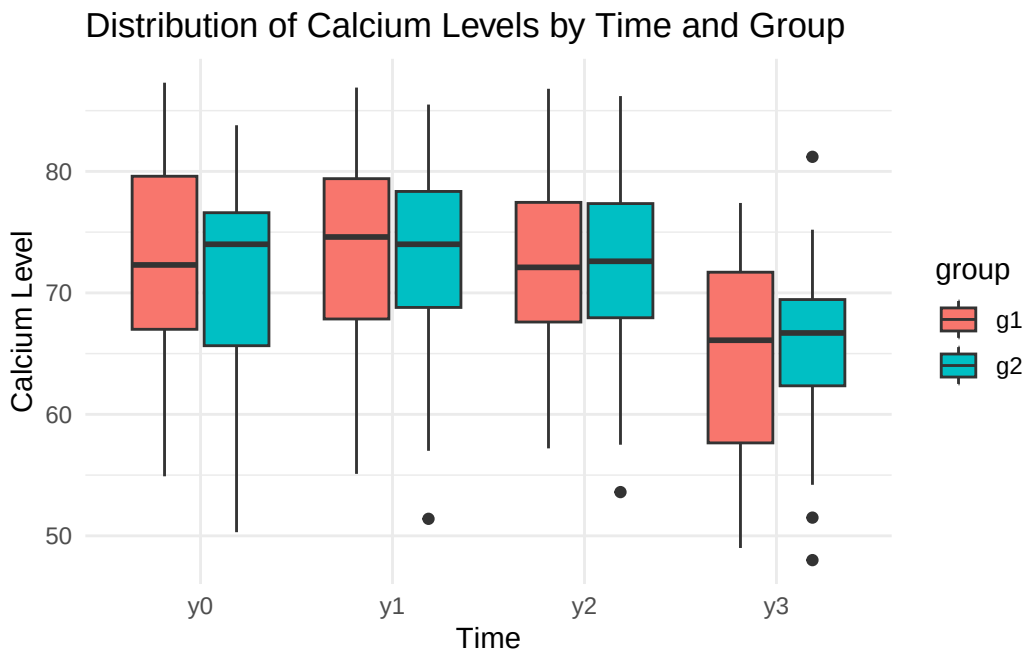
Good—this output tells us exactly what's going on, and it's a bit messier than your original plan assumed.

Right now your data is in wide format with repeated measures (y0, y1, y2, y3) appearing twice (likely two groups or conditions). That means before plotting anything meaningful, you must reshape it. Otherwise, ggplot won't give you a clean comparison.

Here's a revised version of your code that actually fits your dataset structure:

```
New names:
Rows: 16 Columns: 8
-- Column specification
-------------------------------------------------------- Delimiter: "," dbl
(8): y0...1, y1...2, y2...3, y3...4, y0...5, y1...6, y2...7, y3...8
i Use `spec()` to retrieve the full column specification for this data. i
Specify the column types or set `show_col_types = FALSE` to quiet this message.
* `y0` -> `y0...1`
* `y1` -> `y1...2`
* `y2` -> `y2...3`
* `y3` -> `y3...4`
* `y0` -> `y0...5`
* `y1` -> `y1...6`
* `y2` -> `y2...7`
```

```
* `y3` -> `y3...8`
```



What changed from your original plan (and why it matters):

The biggest fix is pivot_longer(). Your plan didn't explicitly account for reshaping, but your dataset requires it. You actually have two groups and four time points, so the analysis becomes a repeated-measures comparison, not just a single-variable plot. A line plot is more informative than a scatterplot here because it shows how each subject changes over time.

6 Appendix B: Code Appendix

```r
library(tidyverse)   # Data wrangling + visualization
library(rvest)       # Web scraping
library(knitr)       # Table creation
library(kableExtra)  # Table styling

# Step 1: Scrape the Wikipedia Page

url <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"

# Read in the HTML from the webpage
page <- read_html(url)

# Extract all tables with wikitable
wiki_tables <- page |>
  html_elements("table.wikitable") |>
  html_table()

# Step 2: Extract Relevant Columns
```

```r
# Column positions are fixed based on current Wikipedia layout

t2025 <- wiki_tables[[1]][, c(2, 5, 6)]
t2024 <- wiki_tables[[2]][, c(2, 5, 6)]
t2023 <- wiki_tables[[3]][, c(2, 5, 6)]
t2022 <- wiki_tables[[4]][, c(2, 5, 6)]
t2021 <- wiki_tables[[5]][, c(2, 5, 6)]
t2020 <- wiki_tables[[6]][, c(2, 5, 6)]

# Step 3: Standardize Column Names

col_names <- c("airport", "code", "passengers")

names(t2025) <- col_names
names(t2024) <- col_names
names(t2023) <- col_names
names(t2022) <- col_names
names(t2021) <- col_names
names(t2020) <- col_names

# Step 4: Add the Year Variable

t2025 <- t2025 |> mutate(year = 2025)
t2024 <- t2024 |> mutate(year = 2024)
t2023 <- t2023 |> mutate(year = 2023)
t2022 <- t2022 |> mutate(year = 2022)
t2021 <- t2021 |> mutate(year = 2021)
t2020 <- t2020 |> mutate(year = 2020)

# Step 5: Combine All Years

tidy_airports <- bind_rows(t2025, t2024, t2023, t2022, t2021, t2020)

# Airports of interest
target_codes <- c("ATL", "FRA", "PKX", "LHR", "DXB", "HND")

# Step 6: Clean and Filter Data

final_airports <- tidy_airports |>
  mutate(
    # Ensure airport codes are exactly 3 characters
    code = substr(code, 1, 3),

    # Remove commas and non-numeric characters, then convert to numeric
    passengers = as.numeric(gsub("[^0-9]", "", passengers)),

    # Convert to millions
```

```r
    passengers_millions = passengers / 1e6
  ) |>
  # Keep only selected airports
  filter(code %in% target_codes) |>
  # Sort by year and passenger volume
  arrange(year, desc(passengers))

# Step 7: Create Summary Table

airport_table_data <- final_airports |>
  # Keep only relevant columns
  select(airport, code, year, passengers_millions) |>
  # Convert from long to wide
  pivot_wider(
    names_from = year,
    values_from = passengers_millions
  ) |>
  arrange(code)

# Render table
airport_table_data |>
  kable(
    caption = "Comparison of Annual Passenger Traffic (Millions) Across Major Global Airports
    digits = 2,
    col.names = c("Airport Name", "IATA Code", "2025", "2024", "2023", "2022", "2021", "2020")
  ) |>
  kable_classic(full_width = FALSE, html_font = "Cambria") |>
  kable_styling(bootstrap_options = c("striped", "hover"))
```

Table 2: Comparison of Annual Passenger Traffic (Millions) Across Major Global Airports (2020–2025)

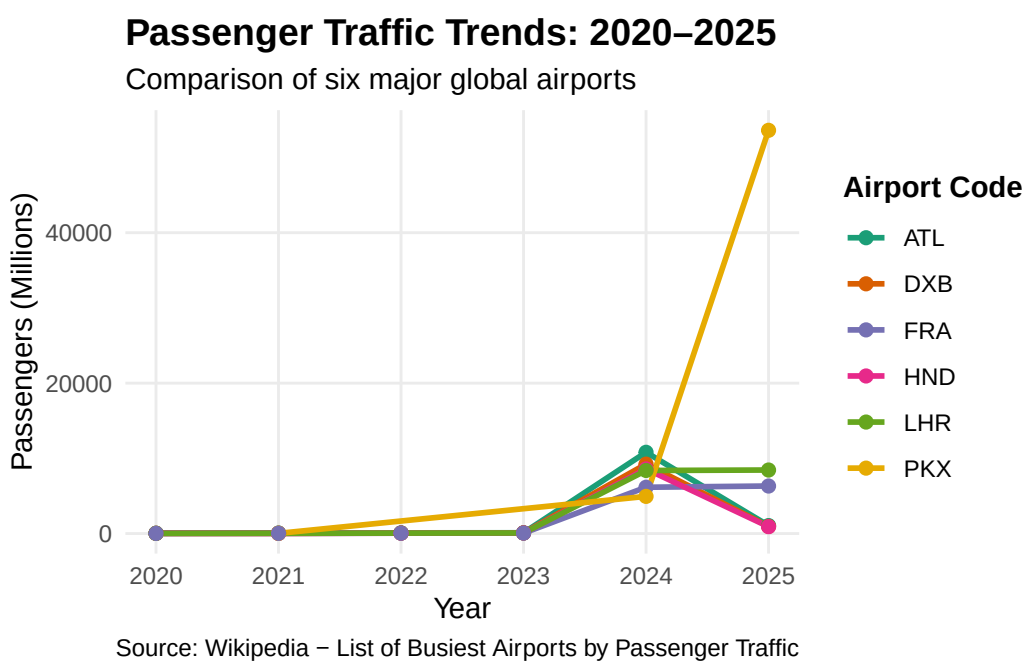
Airport Name	IATA Code	2025	2024	2023	2022	2021	
Hartsfield–Jackson Atlanta International Airport	ATL	42.92	75.70	93.70	104.65	10806.78	10
Dubai International Airport	DXB	25.84	29.11	66.07	86.99	9233.15	9
Frankfurt Airport	FRA	18.77	24.81	48.92	59.36	6156.50	63
Tokyo Haneda Airport	HND	31.06	25.88	50.29	78.72	8590.02	9
Heathrow Airport	LHR	22.11	NA	61.61	79.18	8388.46	84
Beijing Daxing International Airport	PKX	NA	25.05	NA	NA	4944.10	536

```r
# Step 8: Visualization

library(ggplot2)

final_airports |>
```

```r
  ggplot(aes(x = year, y = passengers_millions, color = code, group = code)) +
  geom_line(linewidth = 1) +        # Trend lines
  geom_point(size = 2) +            # Data points
  scale_color_brewer(palette = "Dark2") +
  labs(
    title = "Passenger Traffic Trends: 2020-2025",
    subtitle = "Comparison of six major global airports",
    x = "Year",
    y = "Passengers (Millions)",
    color = "Airport Code",
    caption = "Source: Wikipedia - List of Busiest Airports by Passenger Traffic"
  ) +
  theme_minimal() +
  theme(
    panel.grid.minor = element_blank(),
    plot.title = element_text(face = "bold", size = 14),
    legend.position = "right",
    legend.title = element_text(face = "bold")
  )
```




```r
library(tidyverse)
library(patchwork)

# Step 1: Define a plotting helper function to avoid repeating code 4 times
make_mc_plot <- function(n_res) {
  x_min <- 0; x_max <- 4; y_min <- 0; y_max <- 1
  box_area <- (x_max - x_min) * (y_max - y_min)
```

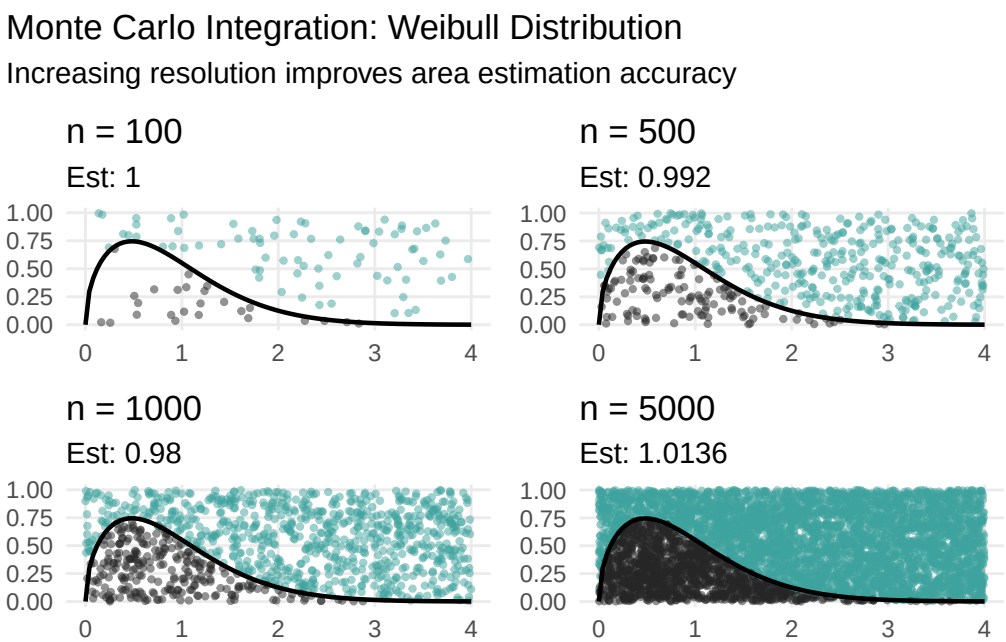
```r
  # Step 1A: Generate and wrangle
  sim_data <- generate_mc_points(n_res, x_min, x_max, y_min, y_max) |>
    mutate(
      f_x = dweibull(x, shape = 1.5, scale = 1),
      status = if_else(y <= f_x, "In", "Out"),
      is_in = if_else(status == "In", 1, 0)
    )

  # Step 1B: Calculate estimate
  est <- mean(sim_data$is_in) * box_area

  # Step 1C: Create plot
  ggplot(sim_data, aes(x = x, y = y)) +
    geom_point(aes(color = status), alpha = 0.5, size = 0.8) +
    stat_function(fun = dweibull, args = list(shape = 1.5, scale = 1),
                  xlim = c(0, 4), color = "black", linewidth = 0.8) +
    scale_color_manual(values = c("In" = "#262626", "Out" = "#3EA39E")) +
    labs(
      title = paste("n =", n_res),
      subtitle = paste("Est:", round(est, 4)),
      x = NULL, y = NULL # Keep it clean for small multiples
    ) +
    theme_minimal() +
    theme(legend.position = "none", panel.grid.minor = element_blank())
}

# Step 2: Create the four plot objects
plot100  <- make_mc_plot(100)
plot500  <- make_mc_plot(500)
plot1000 <- make_mc_plot(1000)
plot5000 <- make_mc_plot(5000)

# Step 3: Create the small multiple using patchwork
(plot100 + plot500) / (plot1000 + plot5000) +
  plot_annotation(title = "Monte Carlo Integration: Weibull Distribution",
                  subtitle = "Increasing resolution improves area estimation accuracy")
```




```r
library(tidyverse)

# Import
calcium <- read_csv("calcium.csv")
```

```
New names:
Rows: 16 Columns: 8
-- Column specification
-------------------------------------------------------- Delimiter: "," dbl
(8): y0...1, y1...2, y2...3, y3...4, y0...5, y1...6, y2...7, y3...8
i Use `spec()` to retrieve the full column specification for this data. i
Specify the column types or set `show_col_types = FALSE` to quiet this message.
* `y0` -> `y0...1`
* `y1` -> `y1...2`
* `y2` -> `y2...3`
* `y3` -> `y3...4`
* `y0` -> `y0...5`
* `y1` -> `y1...6`
* `y2` -> `y2...7`
* `y3` -> `y3...8`
```

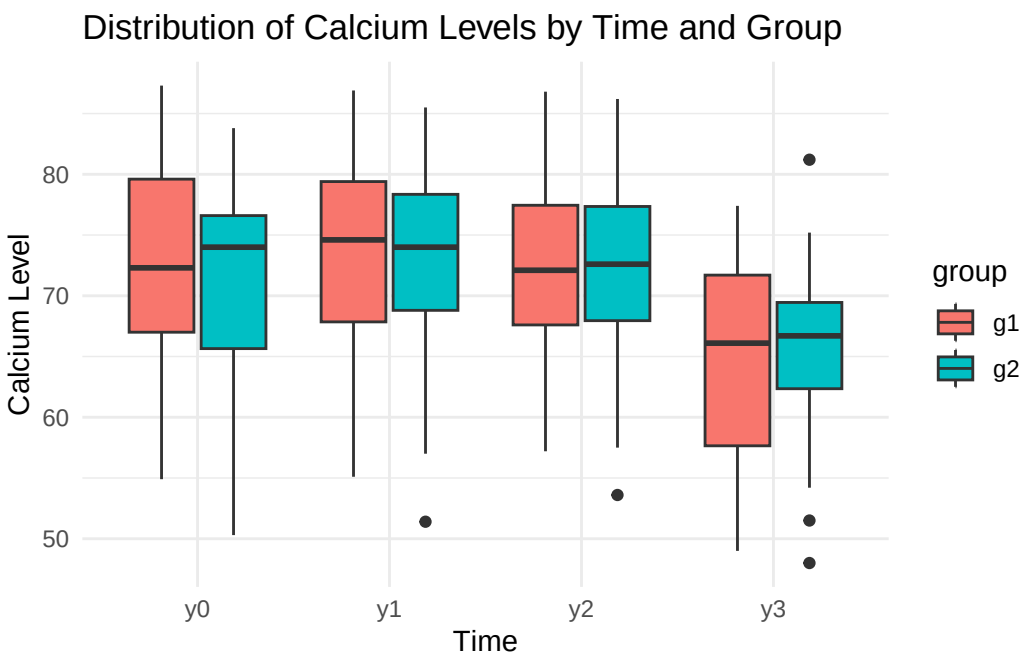
```r
# Step 1: Rename columns
calcium <- calcium |>
  rename(
    g1_y0 = 1, g1_y1 = 2, g1_y2 = 3, g1_y3 = 4,
    g2_y0 = 5, g2_y1 = 6, g2_y2 = 7, g2_y3 = 8
  )
```

```r
# Step 2: Clean (no calcium_level yet!)
calcium_clean <- calcium %>%
  drop_na()

# Step 3: Reshape (this creates calcium_level)
calcium_long <- calcium_clean %>%
  mutate(id = row_number()) %>%
  pivot_longer(
    cols = -id,
    names_to = c("group", "time"),
    names_sep = "_",
    values_to = "calcium_level"
  )

# Step 4: Filter it
calcium_long <- calcium_long %>%
  filter(!is.na(calcium_level)) %>%
  mutate(
    group = factor(group),
    time = factor(time, levels = c("y0", "y1", "y2", "y3"))
  )

# Step 5: Boxplot for distribution comparison
ggplot(calcium_long, aes(x = time, y = calcium_level, fill = group)) +
  geom_boxplot() +
  labs(
    title = "Distribution of Calcium Levels by Time and Group",
    x = "Time",
    y = "Calcium Level"
  ) +
  theme_minimal()
```


Distribution of Calcium Levels by Time and Group
Calcium Level
80
70
60
50
y0
y1
y2
y3
Time
group
g1
g2